**SIMATS ENGINEERING**
**Department of Computer Science and Engineering**
**CSA16 – Data Warehousing and Data Mining**
**ASSIGNMENT 2 – CO2**

## Design, Build and Validate a Data Warehouse and OLAP Reporting Solution for a Retail Chain

CO2: Apply suitable Data Pre-processing techniques like Data cleaning, Data integration, Data transformation and data Reduction for effective data preparation

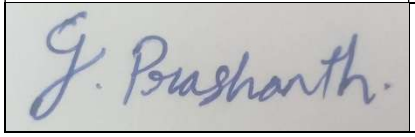
| Student Name | Register Number | Department / Section | Academic Year / Semester | Faculty Name | Nature of Assignment | Duration | Total Marks | Date of Submission | Date of Evaluation |
|---|---|---|---|---|---|---|---|---|---|
| G Prashanth | 192311366 | CSE | 2026–2027 | Dr.D. Beulah David | Individual | 1 Week | 100 | 06/11/2026 | |

## MARK SHEET (Rubric-based Evaluation)

| Task | Task Title | Max | Marks Awarded | Remarks / Evidence Expected |
|---|---|---|---|---|
| Q1 | Mining Functionalities and Algorithm Mapping | 10 | | Six telecom problems mapped to functionality, supervised/unsupervised type, data attributes/source and at least two suitable algorithms; evaluate the all-purpose algorithm proposal. |
| Q2 | KDD Process, Architecture and Mining Issues | 10 | | MediCore KDD mapping; labelled mining-system architecture; classification by four views; at least five issue–mitigation pairs; distinguish mining from SQL and basic statistics. |
| Q3 | Data Dispersion and Outlier Analysis | 14 | | Mean, median, mode, midrange, range, quartiles, IQR, variance and standard deviation; five-number summary, boxplot, outlier test, removal analysis, measure classification and tool verification. |
| Q4 | Data Cleaning and Noise Handling | 12 | | Defect classification and justified handling; equal-frequency binning (depth 4), mean/boundary smoothing; regression/clustering discussion and imputation impact. |
| Q5 | Data Integration and Transformation | 14 | | Conflict-resolution schema; redundancy; Pearson correlation and chi-square calculations; min-max, z-score and decimal scaling; Euclidean distance comparison and transformation examples. |
| Q6 | Data Reduction and Discretization | 14 | | Reduction methods, sampling comparison for rare failures, PCA criterion, equal-width/equal-frequency intervals, entropy/information-gain table and discretization comparison. |
| Q7 | Day-4 Design Challenge – Diagnose, Redesign and Validate | 26 | | A: diagnose pipeline defects (2); B: measurable objectives/constraints (2); C: revised preprocessing pipeline and justification (4); D: predictions (1); E: implementation/validation (2); F: defence and remaining risk (2). |
| | **TOTAL** | **100** | | **Marks to be awarded by the faculty.** |

## OVERALL REMARKS

Faculty feedback:

| Student Signature | Faculty Signature | Program Director |
|---|---|---|
|  | | – |

# MANDATORY SUBMISSION EVIDENCE CHECK

✓ = present  P = partly present  ✗ = missing. Mark the status after checking the student's submitted report and supporting evidence.

| Task | Evidence Expected | Status | Note |
|---|---|---|---|
| Q1 | Six problems mapped to functionality, learning type, data attributes/source and candidate algorithms. | ☐ | All six cases and the design decision addressed. |
| Q2 | KDD activities, architecture diagram, four-view classification and issue–mitigation table. | ☐ | At least five scenario-specific issues; SQL/statistics comparison. |
| Q3 | Manual calculations, five-number summary, boxplot, outliers and tool verification. | ☐ | Quartile/variance conventions stated; removal analysis included. |
| Q4 | Defect classification, handling strategy, binning and smoothing calculations. | ☐ | All bins shown; imputation impact discussed. |
| Q5 | Conflict table, correlation, chi-square expected values and interpretation. | ☐ | Three normalisation methods and distance comparison included. |
| Q6 | Reduction strategy, sampling choice, PCA criterion and discretisation calculations. | ☐ | Intervals and information-gain calculations included. |
| Q7-A/B | Pipeline diagnosis, violated principles, measurable objectives and constraints. | ☐ | All original pipeline defects considered. |
| Q7-C/D | Ordered revised pipeline, alternatives, training-only decisions and predictions. | ☐ | Justifications and predictions documented. |
| Q7-E/F | Implementation comparison, metrics, interpretation, defence and remaining risk. | ☐ | Same test data used; default recall reported. |

# IMPLEMENTATION EVIDENCE CHAIN (Tasks 7–9)

| Evidence Item | Status | Observation |
|---|---|---|
| Manual calculation sheets | ☐ | Formulae, assumptions and intermediate values shown for Q3–Q6. |
| Dataset / data source | ☐ | Dataset description, source or link and relevant attributes documented. |
| Spreadsheet / Python / Weka / KNIME files | ☐ | Files or workflows used for calculation and verification submitted. |
| Tool settings and screenshots | ☐ | Settings, selected attributes, preprocessing and outputs captured. |
| Plots and visual evidence | ☐ | Boxplot, architecture and other required diagrams are labelled and readable. |
| Q7 baseline pipeline | ☐ | Original preprocessing steps and baseline results are reproducible. |
| Q7 redesigned pipeline | ☐ | Ordered steps, fitted parameters and training/test separation documented. |
| Validation and metric comparison | ☐ | Same test set; accuracy and class-sensitive measures, especially default recall, reported. |
| Reproducibility and consistency | ☐ | Dataset, code, calculations, screenshots and report values agree. |

## SUBMISSION CHECKLIST (not separately marked)

| Requirement | Status |
|---|---|
| All Questions Q1–Q7 are answered. | ☐ Yes ☐ Partly ☐ No |
| All six Q1 problems are mapped to mining functionalities and algorithms. | ☐ Yes ☐ Partly ☐ No |
| KDD stages, system architecture and mining issues are explained using the hospital scenario. | ☐ Yes ☐ Partly ☐ No |
| Q3 manual calculations, quartile method, boxplot and outlier analysis are included. | ☐ Yes ☐ Partly ☐ No |
| Q4 defects, cleaning decisions, binning and smoothing calculations are shown. | ☐ Yes ☐ Partly ☐ No |
| Q5 integration conflicts, correlation, chi-square and normalisation calculations are included. | ☐ Yes ☐ Partly ☐ No |
| Q6 reduction methods, sampling, discretisation and information-gain calculations are included. | ☐ Yes ☐ Partly ☐ No |
| Q7 diagnosis, objectives, revised pipeline, predictions, implementation and defence are provided. | ☐ Yes ☐ Partly ☐ No |
| Datasets, spreadsheets, code/workflows and tool screenshots are submitted where applicable. | ☐ Yes ☐ Partly ☐ No |
| Diagrams, tables, calculations and references are numbered, labelled and readable. | ☐ Yes ☐ Partly ☐ No |
| Assumptions, formulas, conventions and sources are clearly stated. | ☐ Yes ☐ Partly ☐ No |
| No unfilled template placeholders remain in the student report. | ☐ Yes ☐ Partly ☐ No |
| Viva recommended where calculations, implementation or results need clarification. | ☐ Yes ☐ No |

Note: This DWDM assignment evaluation template is based on Assignment 2 – CO2. Blank marks and evidence statuses are intentionally left for faculty evaluation.

# Q1 – Matching Business Problems to Mining Functionalities (10 Marks)

**QUESTION**

TeleNova (8 million subscribers) has six business problems: P1 plans/add-ons bought together; P2 will a customer leave next month; P3 natural usage segments with no labels; P4 SIMs with very different calling behaviour; P5 characteristics of high-value customers versus low-value; P6 how monthly data usage changed over 24 months.

For each problem identify: (a) the mining functionality; (b) supervised/unsupervised and the kind of pattern produced; (c) data attributes and source (database or warehouse); (d) at least two candidate algorithms and why. Then evaluate the marketing manager's proposal to buy a single "all-purpose mining algorithm" for all six problems.

## ANSWER

### 1.1 Problem-to-functionality mapping

| # | Functionality | Learning type and pattern produced | Data attributes and source | Candidate algorithms and reason |
|---|---|---|---|---|
| P1 | Association analysis | **Unsupervised.** Produces association rules such as {Plan-X} ⇒ {Add-on-Y} with support and confidence. | Transaction-style table: customer_id, billing_month, plan_id, addon_id. **Source:** data warehouse (sales/billing fact table, history needed). | **Apriori:** simple, finds all frequent itemsets by level-wise search. **FP-growth:** no candidate generation, faster on 8 M subscribers. Chosen because the question is about items occurring together. |
| P2 | Classification (the output is a class label: churn / no churn; predicting a numeric value would be "prediction") | **Supervised** (needs historical churn label). Produces a model: decision rules or probability of churn. | tenure, plan, ARPU, monthly usage trend, recharge frequency, complaints, network issues, contract type, age group + label *churned_next_month*. **Source:** warehouse (historical, labelled snapshots). | **Decision tree (C4.5/ID3):** readable rules for retention team. **Naive Bayes / logistic regression:** fast, gives churn probability, scales to 8 M rows. (Neural network or k-NN are alternatives but less interpretable.) |
| P3 | Clustering | **Unsupervised.** Produces groups (segments) with centroids, e.g. heavy-data youth, voice-only seniors. | Per-subscriber aggregates: minutes, SMS, data MB, night/day ratio, roaming share, top-up value. **Source:** warehouse aggregates (not raw call records). | **k-means:** simple, scales to millions of points. **BIRCH / hierarchical:** handles very large data with a tree summary and shows nested segments. DBSCAN is an option for irregular shapes but costly at 8 M. |
| P4 | Outlier (anomaly) analysis | **Unsupervised** (no labels for "abnormal"). Produces a list of rare, deviating objects with an outlier score. | SIM_id, calls/day, avg duration, distinct numbers called, international share, night-call share, SMS burst counts. **Source:** operational database (call detail records) plus warehouse aggregates for baselines. | **Distance-based / density-based (LOF):** flags points far from neighbours; no distribution assumption. **Statistical (z-score, boxplot 1.5×IQR):** quick and explainable for single attributes. **Clustering-based:** points outside every cluster. |
| P5 | Concept / class description (characterisation + discrimination) | **Class-labelled but descriptive.** The high/low-value classes are given; output is a generalised description (characteristic rule) and a comparison (discriminant rule). | value_class (high/low), age group, region, plan type, tenure, ARPU, data usage. **Source:** warehouse (data cube with concept hierarchies). | **Attribute-oriented induction:** generalises attributes via concept hierarchies to give compact descriptions. **OLAP data cube roll-up/drill-down:** compares classes by dimension. (A decision tree can also give discriminating rules.) |

| # | Functionality | Learning type and pattern produced | Data attributes and source | Candidate algorithms and reason |
|---|---|---|---|---|
| P6 | Evolution analysis (time-related / trend analysis) | **Mainly unsupervised/descriptive** (supervised only if forecasting is added). Produces trends, seasonality, change points. | subscriber_id, month (24 values), data_MB, segment. **Source:** warehouse (time dimension is essential). | **Time-series analysis (moving average, trend/regression, ARIMA):** captures trend and seasonality. **Sequential pattern mining / similarity search (e.g. DTW):** finds subscribers with similar usage trajectories. |

## 1.2 Evaluation of the "single all-purpose algorithm" proposal

**Verdict: the proposal should be rejected.** The analysis above shows the six problems are different kinds of mining tasks, not six variations of one task:

- **Different functionalities and outputs.** The outputs range from rules (P1), a predictive model (P2), groups (P3), a ranked list of exceptions (P4), generalised descriptions (P5) to a trend (P6). No single algorithm produces all of these.
- **Different learning types.** P2 (and, in a class-labelled sense, P5) need labelled history; P1, P3, P4, P6 do not. An algorithm that needs labels cannot cluster, and a clustering algorithm cannot predict churn.
- **Different data shapes.** P1 needs transactions/itemsets, P6 needs ordered time series, P3/P4 need numeric feature vectors, P2/P5 need labelled tuples.
- **Different evaluation measures.** Support/confidence/lift (P1), accuracy, recall, ROC (P2), silhouette/within-cluster variance (P3), precision of flagged anomalies (P4), interpretability (P5), forecast error (P6).
- **Scalability and cost.** With 8 million subscribers an algorithm that is efficient for one task (e.g. FP-growth) is irrelevant for another.

A decision tree is the closest to a general-purpose tool (it can serve P2, P5 and, loosely, P4) but it still cannot do P1, P3 or P6. **Recommendation:** buy or adopt a mining *platform/toolkit* that contains a library of algorithms (association, classification, clustering, outlier and time-series modules) on top of the existing warehouse, and select the algorithm per problem. This also matches the principle that algorithm choice depends on the task, data and objective, not on a single product.

# Q2 – The KDD Process, System Architecture and Mining Issues (10 Marks)

**QUESTION**

MediCore, a hospital network, wants to discover factors that cause patients to be readmitted within 30 days. Data sits in admission, laboratory, pharmacy and billing systems.
(a) Map each KDD step to MediCore activities. (b) Draw the architecture of a data mining system for MediCore and mark where each requirement is handled. (c) Classify the system on four views: kinds of data, kinds of knowledge, techniques used, applications. (d) Identify at least five data mining issues specific to this scenario with a mitigation for each.
Inquiry: A director says "We already have SQL and statistics, so we do not need data mining." Using a MediCore example, explain how a mining task differs from a database query and from a basic statistical summary.

**ANSWER**

## 2.1 KDD steps mapped to MediCore activities

| KDD step | Activities at MediCore | Example output |
|---|---|---|
| 1. Data selection | Define target: *readmitted_within_30_days* (yes/no). Select relevant tables and attributes: admission (age, diagnosis, length of stay, discharge type), laboratory (HbA1c, creatinine), pharmacy (number of drugs, changes at discharge), billing (insurance class, charges). Select the patient cohort (exclude deaths, planned readmissions, transfers). | Target dataset of index admissions with 30-day outcome. |
| 2. Preprocessing (cleaning and integration) | Resolve patient-ID mismatches across four systems; standardise dates and units (mg/dL vs mmol/L); fix inconsistent diagnosis codes; handle missing lab values (class-wise or model-based imputation, not global mean); remove duplicates; treat impossible values (negative stay, age 250) as errors but keep real clinical extremes. | Clean, integrated patient-episode table. |
| 3. Transformation | Derive attributes (readmission flag, number of prior admissions in 12 months, comorbidity count); generalise ICD codes to disease groups and age to bands; normalise numeric lab values; discretise length of stay; reduce dimensions by attribute selection. | Mining-ready feature table. |
| 4. Data mining | Classification (decision tree, logistic regression, Naive Bayes) to predict readmission; association rules (e.g. {diabetes, ≥10 drugs} ⇒ readmit); clustering of high-risk patient groups; outlier analysis for unusual stays. | Models, rules, clusters. |
| 5. Interpretation / evaluation | Measure recall, precision, lift and ROC on held-out data; filter rules by support/confidence/lift; clinicians check plausibility and actionability; remove confounded or obvious patterns (e.g. "ICU stay ⇒ readmit"); present to care managers and feed back into step 1 if needed. | Validated risk factors and a deployed risk score. |

KDD is iterative: poor results in step 5 send the team back to steps 2–4 (for example to re-impute lab values or add features).

## 2.2 Architecture of the MediCore data mining system

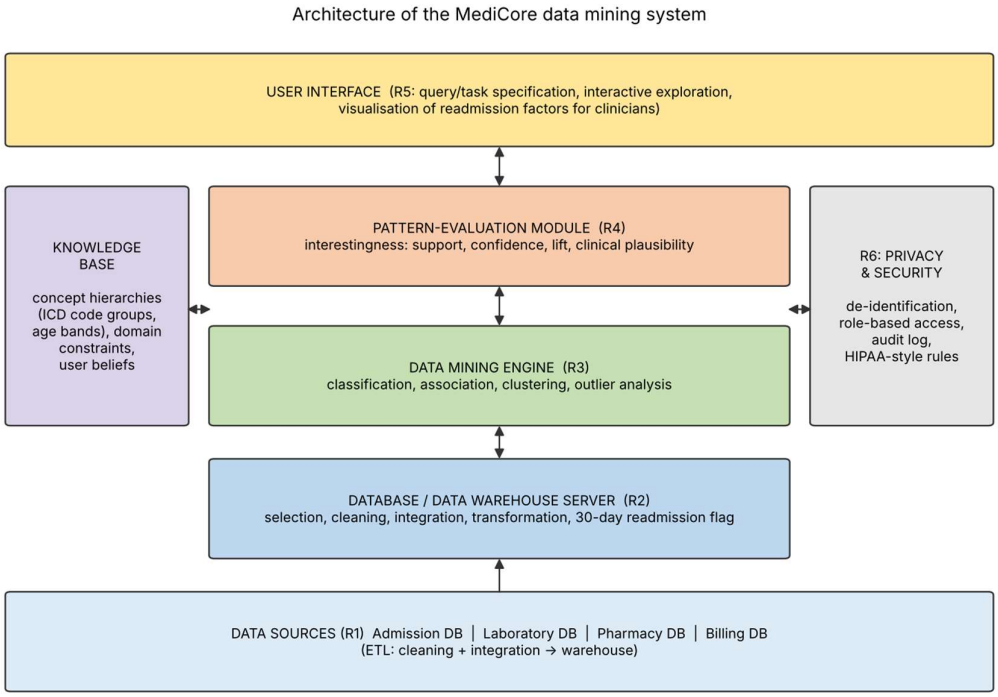



*Figure 1: Data mining system architecture for MediCore (R1–R6 mark where each requirement is handled)*

| Label | Requirement | Handled by | Component |
|---|---|---|---|
| R1 | Data is spread over four systems | ETL/integration with cleaning and conflict resolution | Data sources → warehouse |
| R2 | Fast, consistent, historical access to integrated data | Warehouse server (cube, 30-day flag, patient dimension) | Database/warehouse server |
| R3 | Discover factors causing readmission (classification, association, clustering, outliers) | Algorithm library run on selected data | Mining engine |
| R4 | Only valid, novel, clinically useful patterns | Support/confidence/lift thresholds, clinician rules, domain constraints | Pattern-evaluation module + knowledge base |
| R5 | Clinicians must specify tasks and explore results | Query/task language, visual rule and risk-factor displays | User interface |
| R6 | Patient privacy and security | De-identification, role-based access, audit trail | Cross-cutting security layer |

## 2.3 Classification of the system on four views

| View | Classification for MediCore |
|---|---|
| Kinds of data | Relational/warehouse data (admission, billing), transactional data (pharmacy prescriptions), time-related data (lab results over time); possibly text (discharge notes) later. |
| Kinds of knowledge | Classification and prediction (readmission risk), association rules (conditions and drugs), clustering (patient groups), outlier analysis, concept description (profile of readmitted patients); mining at multiple levels (ICD group vs code). |
| Techniques used | Database/warehouse-oriented, with OLAP and data-cube summarisation; machine-learning techniques (decision trees, Naive Bayes, logistic regression, k-means); statistics for validation; interactive/visualisation techniques for clinicians. |
| Applications | Healthcare analytics: hospital quality improvement, patient risk stratification, resource and discharge planning. |

## 2.4 Data mining issues specific to MediCore, with mitigation

| Issue category | Issue in this scenario | Mitigation |
|---|---|---|
| Mining methodology | Noisy and incomplete data: many missing lab results; patients who got readmitted in another hospital appear as “not readmitted” (label noise). | Imputation by clinical group, missing-value indicators, link with regional discharge data, report sensitivity of results to label noise. |
| Mining methodology | Class imbalance: only a minority of patients are readmitted, so accuracy is misleading. | Stratified sampling, class weights, evaluate with recall, precision, PR-AUC. |
| User interaction | Clinicians need interpretable results and want to inject domain knowledge; black-box scores will not be trusted. | Use decision trees/rules, concept hierarchies (ICD groups), interactive drill-down, show top contributing factors per patient. |
| Efficiency and scalability | Millions of admissions and lab events over many years across hospitals. | Data reduction (aggregation, attribute selection, sampling), FP-growth rather than Apriori, incremental model refresh, parallel processing. |
| Diversity of data types | Structured codes, numeric labs, prescription transactions, free-text notes, time series. | Separate preprocessing per type, common patient key, text features added later, use multi-type feature table. |
| Privacy and social | Sensitive health data; risk of re-identification; bias in risk scores (e.g. by insurance class) and misuse to deny care. | De-identification/k-anonymity, access control and audit logs, consent and legal compliance, fairness audits, use scores only to allocate extra follow-up. |
| Methodology (validity) | Mining finds correlation, not cause; “factors that cause” readmission cannot be proven from observational data. | Phrase results as risk factors, validate with clinicians, consider controlled studies for interventions. |

## 2.5 Reply to the director: “We already have SQL and statistics”

**Query versus mining.** SQL retrieves what is already stored and asked for. At MediCore, *“List all diabetic patients readmitted within 30 days in March”* is a query: the analyst already knows the condition and the answer is a set of existing records. It cannot answer *“which combinations of conditions, drugs, lab values and discharge details are associated with readmission, and what is the risk for a patient being discharged today?”* because the analyst does not know which attributes or combinations to ask for. A mining task searches the space of possible patterns automatically. For instance a decision tree may discover that patients aged over 65 with three or more medication changes at discharge and a discharge on a Friday have a much higher readmission rate, a pattern nobody wrote a query for.

**Statistical summary versus mining.** A basic summary says *“overall 14% of patients are readmitted, mean age 61, mean length of stay 5.2 days”* (illustrative figures). It describes the whole data set one attribute at a time and is mostly confirmatory (tests a hypothesis you already have). Mining works on many attributes at once, finds interactions, produces a model that predicts for new patients, and uses scalable algorithms for millions of records with mixed types and missing values.

**Conclusion.** SQL and statistics are necessary (mining uses them for selection, cleaning and validation) but they answer known questions. Data mining answers unknown questions: it finds hidden patterns and predictive models. The director’s statement is therefore only partly correct.

# Q3 – Describing and Comparing Data Dispersion (14 Marks)

**QUESTION**
Delivery times (minutes) of 24 orders: 28, 31, 33, 35, 35, 36, 38, 40, 41, 42, 42, 43, 45, 46, 48, 50, 52, 53, 55, 58, 61, 64, 72, 95.
(a) Mean, median, mode, midrange and comment on mean vs median. (b) Range, Q1, Q3, IQR, variance, standard deviation (state the quartile convention). (c) Five-number summary, boxplot, 1.5×IQR outliers. (d) Remove the largest value and recompute; which measures changed most and why. (e) Classify mean, median, variance, SD and range as distributive/algebraic/holistic and explain the impact on partitioned warehouse data. (f) Verify with a tool and explain differences.
Inquiry: Which measure to promise as the "typical" delivery time, and what else to report?

**ANSWER**

## 3.1 Central tendency (n = 24)

$\Sigma x = 1143$, so **mean = 1143 / 24 = 47.625 min**.

Median: n is even, so average the 12th and 13th ordered values: (43 + 45) / 2 = **44 min**.

Mode: 35 and 42 each appear twice (all others once), so the data is **bimodal (35 and 42)**.

Midrange = (min + max) / 2 = (28 + 95) / 2 = **61.5 min**.

**Mean versus median.** Mean (47.6) > median (44), so the distribution is **positively (right) skewed**: a few very slow orders (72, 95) pull the mean upward while the middle of the data stays near 44. The midrange (61.5) is even more distorted because it uses only the two extreme values.

## 3.2 Dispersion

**Range** = 95 − 28 = **67 min**.

**Quartile convention:** median of halves (Tukey/textbook method). Because n = 24 is even, the lower half is the first 12 values and the upper half the last 12.

- Lower half: 28, 31, 33, 35, 35, 36, 38, 40, 41, 42, 42, 43 → Q1 = (36 + 38)/2 = **37**
- Upper half: 45, 46, 48, 50, 52, 53, 55, 58, 61, 64, 72, 95 → Q3 = (53 + 55)/2 = **54**
- **IQR** = Q3 − Q1 = 54 − 37 = **17**

**Variance and standard deviation.** $\Sigma x^2 = 59{,}555$. Sum of squared deviations $SS = \Sigma x^2 - (\Sigma x)^2/n = 59{,}555 - 1143^2/24 = 59{,}555 - 54{,}435.375 =$ **5,119.625**.

- Sample variance $s^2 = SS/(n-1) = 5{,}119.625 / 23 =$ **222.59 min²**; $s =$ **14.92 min**
- Population variance $\sigma^2 = SS/n = 5{,}119.625 / 24 = 213.32$ min²; $\sigma = 14.61$ min

The sample formula (n − 1) is used as the main answer because the 24 orders are a sample from the store's deliveries; the population value is given for completeness.

## 3.3 Five-number summary, boxplot and outliers

| Minimum | Q1 | Median | Q3 | Maximum |
|---|---|---|---|---|
| 28 | 37 | 44 | 54 | 95 |

1.5 × IQR rule: 1.5 × 17 = 25.5. Lower fence = 37 − 25.5 = **11.5**; upper fence = 54 + 25.5 = **79.5**. The value **95 > 79.5 is an outlier**. The value 72 is below 79.5, so it is not an outlier; no value is below 11.5. The whisker therefore ends at 72.




*Figure 2: Boxplot of the 24 delivery times (95 flagged as outlier)*

## 3.4 Recomputation after removing the largest value (95), n = 23

$\Sigma x = 1143 - 95 = 1048$; $\Sigma x^2 = 59{,}555 - 9{,}025 = 50{,}530$; $SS = 50{,}530 - 1048^2/23 = 50{,}530 - 47{,}752.35 = 2{,}777.65$. Median is the 12th value = 43. Quartiles are the medians of the lower 11 and upper 11 values (median excluded): Q1 = 6th value = 36, Q3 = 6th value of 45…72 = 53.

| Measure | With 95 (n = 24) | Without 95 (n = 23) | Change | % change |
|---|---|---|---|---|
| Mean | 47.625 | 45.565 | −2.06 | −4.3% |
| Median | 44 | 43 | −1 | −2.3% |
| Mode | 35 and 42 | 35 and 42 | none | 0% |
| Midrange | 61.5 | 50 | −11.5 | −18.7% |
| Range | 67 | 44 | −23 | −34.3% |
| Q1 / Q3 | 37 / 54 | 36 / 53 | −1 / −1 | small |
| IQR | 17 | 17 | 0 | 0% |
| Variance (sample) | 222.59 | 126.26 | −96.33 | −43.3% |
| Std deviation (sample) | 14.92 | 11.24 | −3.68 | −24.7% |
| Five-number summary | 28, 37, 44, 54, 95 | 28, 36, 43, 53, 72 | | |
| Outliers (1.5×IQR) | 95 | none (fences 10.5 and 78.5) | | |

**What changed most and why.** Variance (−43%), range (−34%) and midrange (−19%) changed most because they depend on the extreme value: the range and midrange use the maximum directly and the variance squares the deviation of 95 from the mean (95 contributes $(95 - 47.6)^2 \approx 2{,}244$ of the 5,120 total, i.e. 44% of the sum of squares). Standard deviation changed less because the square root dampens it. The median, mode, Q1, Q3 and IQR hardly changed (robust measures) because they depend on the position of values and not on how extreme the last value is.

## 3.5 Distributive, algebraic and holistic measures

| Measure | Category | Why | Computing across many warehouse partitions |
|---|---|---|---|
| Range | Distributive | max − min; both max and min are distributive. | Each partition sends (min, max); combine with global min and max. Exact. |
| Mean | Algebraic | = sum/count; sum and count are distributive. | Each partition sends (sum, count); mean = Σsum / Σcount. Exact (not the mean of partition means unless sizes are equal). |

| Measure | Category | Why | Computing across many warehouse partitions |
|---|---|---|---|
| Variance | Algebraic | From count, sum and sum of squares (all distributive). | Each partition sends (n, Σx, Σx²); combine, then variance = Σx²/N − mean². Exact. |
| Standard deviation | Algebraic | Square root of variance. | Same as variance, then take the square root. |
| Median (also mode, quartiles) | Holistic | No fixed-size sub-summary can give the exact result; needs the whole ordered data. | Median of partition medians is **wrong**. Need all values (or a merge-sort of partitions) for the exact answer, or use an approximation such as histograms or quantile sketches. |

So in a partitioned warehouse, distributive and algebraic measures can be computed in parallel with small messages, whereas holistic measures are costly or only approximate.

## 3.6 Tool verification (Python / NumPy)

```
import numpy as np
d=[28,31,33,35,35,36,38,40,41,42,42,43,45,46,48,50,52,53,55,58,61,64,72,95]
a=np.array(d)
print(a.mean(), np.median(a), (a.min()+a.max())/2, a.max()-a.min())
print(np.percentile(a,[25,75]))        # linear interpolation
print(a.var(ddof=1), a.std(ddof=1), a.var(), a.std())
```

| Quantity | Manual | Tool output | Difference? |
|---|---|---|---|
| Mean / median / midrange / range | 47.625 / 44 / 61.5 / 67 | 47.625 / 44.0 / 61.5 / 67 | None |
| Mode | 35 and 42 | 35 and 42 (statistics.multimode) | None; NumPy has no mode function, SciPy returns only the smallest mode (35). |
| Q1, Q3, IQR | 37, 54, 17 | 37.5, 53.5, 16 (NumPy/Excel QUARTILE.INC); 36.5, 54.5 (QUARTILE.EXC) | **Yes** – different quartile conventions (median-of-halves vs linear interpolation vs (n+1)p). |
| Variance, SD | 222.59, 14.92 (sample) | 222.59, 14.92 with ddof=1; 213.32, 14.61 with default ddof=0 | None if the same denominator is used; NumPy defaults to population. |
| Outlier | 95 | 95 | None: with IQR = 16 the upper fence is 77.5, 95 is still an outlier. |

Differences arise only from convention (quartile method, n versus n − 1), not from calculation errors; this is why the convention must always be stated.

## 3.7 Inquiry: what “typical” delivery time to promise?

**Recommendation: use the median (44 minutes)** as the typical time. The data is right-skewed with an outlier, so the mean (47.6) is inflated by a few very late orders and the mode is ambiguous (two modes). The median is robust: removing 95 moved it by only 1 minute.

**Report in addition:** (1) the IQR or the range 37–54 min to show the middle 50% of orders, and (2) an upper percentile such as the **90th percentile ≈ 63 min** (4 of 24 orders, about 1 in 6, took more than 60 minutes). A typical value alone hides the late deliveries that cause complaints; a customer promise (e.g. “about 45 minutes, almost always within an hour”) should be set on the

median plus the tail percentile. The outlier (95 min) should be investigated separately (traffic, stock-out, rider shortage) rather than deleted.

# Q4 – Cleaning Noisy and Incomplete Data (12 Marks)

**QUESTION**
Smart-meter extract (Meter, Household type, kWh, Voltage, Region): M101 Apartment 5 230 North | M102 Apartment (missing) 231 North | M103 House 38 229 north | M104 House −14 232 South | M105 (missing) 19 2300 South | M106 Apartment 11 230 S.
Sorted kWh values of 12 other readings: 5, 9, 11, 14, 18, 19, 23, 26, 27, 30, 33, 38.
(a) Identify and classify every defect. (b) Propose and justify a handling strategy for each from: ignore tuple, fill manually, global constant, attribute mean, class-wise mean, most probable value. (c) Equal-frequency binning (depth 4), smoothing by bin means and by bin boundaries. (d) Explain regression and clustering for noise, and why 2300 V must be treated differently from 5 kWh.
Inquiry: effect of replacing every missing value with the global mean on dispersion and later mining quality, and when to avoid it.

**ANSWER**

## 4.1 Defect identification and classification

| Meter | Defect | Class | Explanation |
|---|---|---|---|
| M102 | kWh is blank | Missing value | Consumption not recorded (sensor/communication failure). |
| M105 | Household type is blank | Missing value | Categorical attribute missing. |
| M103 | Region “north” | Inconsistency | Same value as “North” written with different case; would create a false extra category. |
| M106 | Region “S.” | Inconsistency | Abbreviated/different format of “South”. |
| M104 | kWh = −14 | Invalid value | Energy consumed in an hour cannot be negative (sign error or meter fault; net-metering would need a separate attribute). |
| M105 | Voltage = 2300 V | Noise (invalid / outlier value) | About 10 × the normal 230 V of all other meters; almost certainly an extra digit (data-entry/transmission error). Physically impossible for a household supply. |
| M103 | kWh = 38 for a house | **Not a defect** | High but plausible and within the range of the other 12 readings (5–38); checked and kept. |
| M101 | kWh = 5 | **Not a defect** | Low but valid reading (same value appears in the reference set). |

## 4.2 Handling strategy for each defect

| Defect | Strategy | Justification |
|---|---|---|
| M102: kWh missing | **Class-wise mean** (mean of apartments); better: most probable value by regression on hour-of-day, previous readings | Consumption depends strongly on household type, so the apartment mean (from the extract: (5 + 11)/2 = 8 kWh; in practice computed on all apartment readings) is more accurate than the global mean. Ignoring the tuple would lose a valid voltage and region. |
| M105: household type missing | **Global constant “Unknown”** now; **most probable value** (classifier using kWh, region, previous usage) when enough data exists | A categorical value cannot be averaged. With only 19 kWh the type is ambiguous (apartment readings 5 and 11, house readings 38). Using a constant avoids inventing information and keeps the tuple. |
| M103: region “north” | **Fix by rule/dictionary (manual mapping)** → “North” | A deterministic standardisation (case-fold, trim) is exact; no estimation is needed. |
| M106: region “S.” | **Fill manually via mapping table** → “South” | Domain knowledge gives the exact value; build a mapping (S., S, south → South). |

| Defect | Strategy | Justification |
|---|---|---|
| M104: kWh = −14 | Treat as missing, then **class-wise mean** (houses) or fill manually if the source log shows the true value (e.g. a sign error would give 14) | The value is impossible, so it must not be used in averages. Correcting from the original meter log is best; otherwise impute. Only one field of the tuple is bad, so the tuple should not be discarded. |
| M105: voltage = 2300 | **Fill manually** → 230 (decimal/extra-digit error confirmed from meter log); otherwise set to missing and use the **attribute mean** (≈ 230) | Voltage is nearly constant across meters (229–232), so the attribute mean is an excellent estimate. Because this tuple has two defects (type missing, voltage wrong) and kWh is still valid, keep it rather than ignore the tuple. |

## 4.3 Equal-frequency (equi-depth) binning, depth = 4

12 sorted values ÷ 4 per bin = 3 bins.

| Bin | Values | Bin mean | Smoothing by means | Smoothing by boundaries |
|---|---|---|---|---|
| Bin 1 | 5, 9, 11, 14 | 39/4 = 9.75 | 9.75, 9.75, 9.75, 9.75 | 5, 5, 14, 14 |
| Bin 2 | 18, 19, 23, 26 | 86/4 = 21.5 | 21.5, 21.5, 21.5, 21.5 | 18, 18, 26, 26 |
| Bin 3 | 27, 30, 33, 38 | 128/4 = 32 | 32, 32, 32, 32 | 27, 27, 38, 38 |

**Boundary smoothing rule:** each value is replaced by the closer of the bin's minimum and maximum. Bin 1: 9 is 4 from 5 and 5 from 14 → 5; 11 is 6 from 5 and 3 from 14 → 14. Bin 2: 19 is 1 from 18 → 18; 23 is 3 from 26 (5 from 18) → 26. Bin 3: 30 is 3 from 27 (8 from 38) → 27; 33 is 6 from 27 and 5 from 38 → 38.

**Smoothed data (means):** 9.75 ×4, 21.5 ×4, 32 ×4. **Smoothed data (boundaries):** 5, 5, 14, 14, 18, 18, 26, 26, 27, 27, 38, 38.

## 4.4 Regression, clustering and the 2300 V versus 5 kWh question

**Regression** fits a function such as kWh = f(hour of day, household type, temperature). Readings that lie far from the fitted line (large residuals) are suspected noise; they can be replaced by the fitted value (regression smoothing) or flagged. The same model also supplies the most probable value for missing readings.

**Clustering** groups similar readings (e.g. consumption and voltage patterns). Values that fall outside all clusters, or far from their cluster centroid, are outliers/noise candidates; they can be investigated, removed or replaced by the cluster centre.

**Why 2300 V is treated differently from 5 kWh:**

- 5 kWh lies inside the valid range, is consistent with other readings (the reference set contains it) and is physically plausible. It is, at most, a low but **legitimate extreme** that carries information (a small apartment, a vacant home). Smoothing or removing it would distort the real distribution.
- 2300 V violates a domain constraint: household supply is about 230 V (± 10%), and every other meter reads 229–232 V. The value is 10× too large, so it is an **error**, not a rare event. It should be corrected or set to missing, not kept or merely smoothed.
- The principle: statistical outlier detection only *suggests* candidates; domain rules decide whether an outlier is an error (clean it) or a true rare event (keep it).

## 4.5 Inquiry: impact of replacing every missing value with the global mean

**Effect on dispersion.** The mean is preserved but the variance shrinks, because every imputed value has zero deviation from the mean. If a fraction m of values is imputed, the variance falls by roughly a factor (1 − m). Example from the 12 clean readings (mean 21.08, sample variance 103.72, SD 10.18): adding four missing readings filled with 21.08 gives n = 16, variance 76.06 (−27%), SD 8.72 (−14%), while the mean stays 21.08. The distribution gets an artificial spike at the mean, the IQR narrows and the tails look lighter.

**Effect on later mining.**

- Correlations and regression slopes with other attributes are weakened (imputed values ignore household type, hour, region).
- Clustering produces a false dense cluster at the mean and misses true segments; distance-based methods see too many “average” points.
- Classification trees may split on the artificial value; class-conditional distributions (Naive Bayes) are distorted; confidence intervals and tests are too optimistic.
- Genuine outliers appear more extreme relative to the shrunken spread, so outlier detection is biased.

**When to avoid it:** when many values are missing (more than about 5–10% of the attribute); when the data is skewed or has outliers (the mean is not typical; use median or model-based values); when values are not missing at random (e.g. high-usage meters fail more often); when the attribute is strongly related to a class or other attributes (use class-wise mean, regression or kNN imputation); and in tasks sensitive to variance or distance (clustering, outlier detection, regression). A missing-value indicator column or multiple imputation can also be used. Global mean is acceptable only as a quick baseline for a small share of missing values in a roughly symmetric attribute.

# Q5 – Integration and Transformation of Data (14 Marks)

**QUESTION**
A university merges its Admissions DB (student_id, name, dob, hostel_status, scholarship) with its LMS (sid, full_name, attendance_pct, marks, scholarship_flag).
Part A: identify entity-identification, naming, unit/format and value conflicts and resolve them in an integrated schema; explain redundancy and how correlation analysis detects it. Part B: Pearson correlation of attendance {60, 70, 75, 80, 85, 90} and marks {52, 58, 66, 70, 79, 85}; $\chi^2$ test of hostel status versus scholarship (table of counts, critical value 3.84, $\alpha = 0.05$). Part C: normalise incomes 20,000; 35,000; 50,000; 80,000; 150,000 by min-max, z-score, decimal scaling; Euclidean distance of (19, 35,000) and (21, 80,000) before/after min-max (age range 17–25, income 20,000–150,000); one example each of aggregation, generalisation and attribute construction.

## ANSWER

## Part A – Integration

### A1. Conflicts and their resolution

| Conflict type | Example in the two sources | Resolution in the integrated schema |
|---|---|---|
| Entity identification | student_id (Admissions) vs sid (LMS): are they the same key? Different ID formats (e.g. "S001" vs "1") or reused IDs; name spellings differ. | Use metadata (data dictionary) to confirm both identify the same student; standardise the ID format and match on student_id; for unmatched rows use a secondary match (name + dob) and review manually. Integrated key: **student_id**. |
| Naming | name vs full_name; scholarship vs scholarship_flag; student_id vs sid. | Choose one name per concept: **student_id, full_name, scholarship_status**. |
| Unit / format | dob in DD/MM/YYYY in one system and MM-DD-YYYY elsewhere; names as "Last, First" vs "First Last"; attendance as 0–1 fraction vs percent; marks out of 50 vs 100. | dob → ISO date (YYYY-MM-DD); name → "First Last"; **attendance_pct** stored as 0–100; **marks** converted to percentage of maximum. |
| Value | hostel_status coded H/D vs "Hostel/Day scholar"; scholarship as Y/N vs scholarship_flag 1/0; same student shows scholarship = Y in Admissions but flag 0 in LMS. | Common domain: hostel_status ∈ {Hostel, Day scholar}; scholarship_status ∈ {Yes, No}. For true contradictions use a precedence rule (Admissions is system of record for scholarship; LMS for attendance and marks) and log them for manual review. |

**Integrated schema:** Student(student_id PK, full_name, dob [ISO], hostel_status, scholarship_status, attendance_pct [0–100], marks_pct [0–100]).

### A2. Redundancy and correlation analysis

Integration often creates redundant attributes because (i) the same attribute exists under different names in different sources (scholarship / scholarship_flag), (ii) one attribute can be derived from another (age from dob, total marks from component marks), and (iii) the same fact is stored at different granularity. Redundancy wastes storage, slows mining, and can bias algorithms (multicollinearity, double-counting).

**Correlation analysis** detects it: for two numeric attributes the Pearson coefficient r ($|r|$ close to 1 means one is almost a linear function of the other); for two nominal attributes the $\chi^2$ test of independence. A very strong correlation or a significant $\chi^2$ flags a candidate redundant attribute, which can then be dropped or merged after checking the semantics. Correlation does not imply causation.

## Part B – Redundancy analysis

### B1. Pearson correlation: attendance (A) vs marks (M)

Mean of A = 460/6 = 76.67; mean of M = 410/6 = 68.33.

| A | M | A − 76.67 | M − 68.33 | (A−Ā)(M−M̄) | (A−Ā)² / (M−M̄)² |
|---|---|---|---|---|---|
| 60 | 52 | −16.67 | −16.33 | 272.22 | 277.78 / 266.78 |
| 70 | 58 | −6.67 | −10.33 | 68.89 | 44.44 / 106.78 |
| 75 | 66 | −1.67 | −2.33 | 3.89 | 2.78 / 5.44 |
| 80 | 70 | 3.33 | 1.67 | 5.56 | 11.11 / 2.78 |
| 85 | 79 | 8.33 | 10.67 | 88.89 | 69.44 / 113.78 |
| 90 | 85 | 13.33 | 16.67 | 222.22 | 177.78 / 277.78 |
| **Sum** | | | | **661.67** | **583.33 / 773.33** |

$r = \Sigma(A - \bar{A})(M - \bar{M}) / \sqrt{[\Sigma(A - \bar{A})^2 \times \Sigma(M - \bar{M})^2]} = 661.67 / \sqrt{(583.33 \times 773.33)} = 661.67 / 671.65 =$ **0.985**.

**Interpretation:** a very strong positive linear relationship ($r^2 = 0.97$, so about 97% of the variation in marks is linearly associated with attendance). Students with higher attendance score higher marks. This shows association, not proof of causation, and it is based on only six students. For redundancy: attendance and marks are different concepts, so both are kept; but for a predictive model one carries much of the information of the other (multicollinearity), so regularisation or dropping one may be considered.

### B2. $\chi^2$ test: hostel status vs scholarship

$H_0$: hostel status and scholarship are independent. Expected count = (row total × column total) / 700.

| Cell | Observed (O) | Expected (E) | (O − E)² / E |
|---|---|---|---|
| Hostel, Scholarship | 120 | 300 × 200 / 700 = 85.71 | (34.29)² / 85.71 = 13.71 |
| Hostel, No scholarship | 180 | 300 × 500 / 700 = 214.29 | (−34.29)² / 214.29 = 5.49 |
| Day scholar, Scholarship | 80 | 400 × 200 / 700 = 114.29 | (−34.29)² / 114.29 = 10.29 |
| Day scholar, No scholarship | 320 | 400 × 500 / 700 = 285.71 | (34.29)² / 285.71 = 4.11 |
| **Total** | 700 | 700 | **$\chi^2$ = 33.6** |

Degrees of freedom = (2 − 1)(2 − 1) = 1; critical value at 0.05 = 3.84. Since **33.6 > 3.84, $H_0$ is rejected**: hostel status and scholarship are **not independent** (they are statistically associated at the 0.05 level). Hostel residents receive scholarships at a rate of 120/300 = 40%, against 80/400 = 20% for day scholars. The attributes are correlated but not identical concepts, so both are kept; the result is an association (not causation) and is useful as a mining feature.

## Part C – Transformation

### C1. Normalisation of annual family income

Values: 20,000; 35,000; 50,000; 80,000; 150,000. min = 20,000, max = 150,000, mean = 335,000/5 = 67,000. Population SD: squared deviations (−47,000, −32,000, −17,000, 13,000, 83,000)² sum to 10,580 × 10⁶, divided by 5 = 2,116 × 10⁶, so σ = 46,000.

| Income (₹) | Min-max (0–1) | Z-score (σ = 46,000) | Z-score (sample SD 51,430) | Decimal scaling (j = 6) | Formula note |
|---|---|---|---|---|---|
| 20,000 | 0.000 | −1.022 | −0.914 | 0.020 | v' = (v − 20,000)/130,000 |
| 35,000 | 0.115 | −0.696 | −0.622 | 0.035 | z = (v − 67,000)/σ |
| 50,000 | 0.231 | −0.370 | −0.331 | 0.050 | v' = v / $10^j$ |

| Income (₹) | Min-max (0–1) | Z-score (σ = 46,000) | Z-score (sample SD 51,430) | Decimal scaling (j = 6) | Formula note |
|---|---|---|---|---|---|
| 80,000 | 0.462 | 0.283 | 0.253 | 0.080 | j = smallest integer with max\|v'\| < 1 |
| 150,000 | 1.000 | 1.804 | 1.614 | 0.150 | 150,000/10⁵ = 1.5 ≥ 1, so j = 6 |

### C2. Euclidean distance before and after normalisation

Students: X = (19, 35,000) and Y = (21, 80,000).

**Before:** d = √[(21 − 19)² + (80,000 − 35,000)²] = √(4 + 2,025,000,000) ≈ **45,000**.

**After min-max:** age: (19 − 17)/8 = 0.25 and (21 − 17)/8 = 0.50; income: (35,000 − 20,000)/130,000 = 0.1154 and (80,000 − 20,000)/130,000 = 0.4615. d = √[(0.50 − 0.25)² + (0.4615 − 0.1154)²] = √(0.0625 + 0.1198) = √0.1823 = **0.427**.

**What it reveals:** before normalisation the income difference completely dominates the distance (the age term contributes 4 out of about 2 billion, effectively 0%), so age is ignored. After normalisation both attributes are on the same 0–1 scale and age contributes about 34% of the squared distance, income 66%. Distance-based methods (k-NN, k-means, hierarchical clustering, LOF) need normalised attributes, otherwise the attribute with the largest numeric range decides the result.

### C3. Examples of aggregation, generalisation and attribute construction

- **Aggregation:** daily attendance records summarised to monthly or semester attendance_pct per student; or students aggregated to department-level average marks.
- **Generalisation:** dob → age → age group (17–19, 20–22); marks_pct → grade (A, B, C); a specific hostel block → hostel/day scholar (concept hierarchy climbing).
- **Attribute construction:** age_at_admission computed from dob; or engagement_score = 0.5 × attendance_pct + 0.5 × marks_pct; or income_per_family_member.

# Q6 – Data Reduction and Discretization (14 Marks)

**QUESTION**

A factory logs 200 sensor attributes, 5 million rows per month; analysts need faster mining without losing the ability to detect machine failure.
Part A: describe one technique per category and when it suits this data (data-cube aggregation; attribute subset selection; PCA; numerosity reduction; compression); compare SRSWOR, SRSWR, cluster and stratified sampling and choose one if only 0.5% of rows are failures; explain how many principal components to keep and the criterion.
Part B: temperatures 3, 7, 7, 9, 12, 14, 14, 15, 18, 21, 21, 22, 25, 25, 27, 31, 36, 36, 40, 44 → 4 intervals by equal-width and equal-frequency; entropy-based best split for labelled values (10 N, 12 N, 15 N, 18 N, 22 Y, 25 Y, 28 N, 31 Y, 35 Y, 40 Y) with information gain of every candidate; compare unsupervised and supervised discretization.

## ANSWER

## Part A – Choosing reduction strategies

### A1. One technique per category

| Category | Technique | When it suits the sensor data |
|---|---|---|
| Data-cube aggregation | Roll up raw readings to hour-level per machine (min, max, mean, SD per sensor) along the time and machine hierarchy. | Suitable when failures develop over minutes/hours; reduces 5 M rows massively. Keep max/min/SD, otherwise short spikes that signal failure are lost. |
| Attribute subset selection | **Forward selection** (start with empty set, add the best attribute each step), **backward elimination** (start with all 200, drop the worst), **decision-tree induction** (attributes in the tree are kept). | With 200 attributes many are irrelevant or duplicated sensors. Decision-tree induction is cheap on millions of rows and handles non-linear failure signs; forward selection is good when few sensors matter. Keeps interpretability (which sensor fails). |
| Dimensionality reduction | **PCA:** project standardised sensors onto a few orthogonal components that capture most variance. | Suitable because physical sensors (temperature, vibration, current) are strongly correlated and numeric. Cost: components are hard to interpret and low-variance directions might contain failure signals. |
| Numerosity reduction | **Sampling** (stratified), **histograms** (store bucket counts instead of raw values), **clustering** (replace rows with cluster centres + counts). | Sampling cuts rows directly; histograms suit per-sensor summaries; clustering suits repeated normal operating states (but rare failure points must not be merged into normal clusters). |
| Compression | Lossless (gzip, run-length) or lossy (wavelet transform, PCA). | Sensor signals are smooth, so wavelets keep spikes with few coefficients; useful for storage and transmission; mining on wavelet/PCA-coefficients is possible. |

### A2. Sampling methods and the 0.5% failure case

| Method | How it works | Behaviour for rare failures (0.5%) |
|---|---|---|
| SRS without replacement (SRSWOR) | Each row can be chosen at most once, equal probability. | Simple, but a 1% sample of 5 M rows contains only about 250 failure rows (variable); rare failure types may be missed. |
| SRS with replacement (SRSWR) | A row may be drawn more than once. | Same issue; duplicates add no information. Rarely needed. |
| Cluster sampling | Select whole groups (e.g. a machine or a day) at random. | Cheap and convenient, but if the selected machines had no failures the sample misses them; failures are clumped. |
| Stratified sampling | Split into strata (e.g. failure / normal, or machine type) and sample within each. | **Appropriate:** keep all (or most) failure rows and a small percentage of normal rows; guarantees failure representation. Use sampling weights when estimating overall rates. |

**Choice: stratified sampling** by failure class (and machine type), e.g. 100% of the roughly 25,000 monthly failure rows and 5–10% of normal rows. The model must then be evaluated on an untouched, non-resampled test set, with class weights or probability correction.

### A3. Number of principal components

Standardise all 200 sensors (z-score), compute PCA on the training data only, and choose k by: (1) **cumulative explained variance** of at least 90–95%; (2) the **scree-plot elbow**; (3) Kaiser's rule (eigenvalue > 1 on standardised data) as a cross-check. Because PCA is unsupervised, finish by comparing the failure-detection recall (cross-validated) for several k and keep the smallest k that does not reduce recall. Because many sensors are correlated, a small fraction of the 200 (typically tens, not hundreds) is expected to be enough; the exact k must come from the data.

## Part B – Discretization by calculation

### B1. Equal-width binning (4 intervals)

min = 3, max = 44, width = (44 − 3)/4 = 10.25.

| Interval | Range | Values | Count |
|---|---|---|---|
| I1 | [3, 13.25) | 3, 7, 7, 9, 12 | 5 |
| I2 | [13.25, 23.5) | 14, 14, 15, 18, 21, 21, 22 | 7 |
| I3 | [23.5, 33.75) | 25, 25, 27, 31 | 4 |
| I4 | [33.75, 44] | 36, 36, 40, 44 | 4 |

### B2. Equal-frequency binning (4 intervals)

20 values ÷ 4 = 5 values per bin.

| Bin | Interval (min–max) | Values | Count |
|---|---|---|---|
| B1 | [3, 12] | 3, 7, 7, 9, 12 | 5 |
| B2 | [14, 21] | 14, 14, 15, 18, 21 | 5 |
| B3 | [21, 27] | 21, 22, 25, 25, 27 | 5 |
| B4 | [31, 44] | 31, 36, 36, 40, 44 | 5 |

Note: the value 21 occurs twice and is split between B2 and B3 by the strict depth-5 rule. If identical values must stay in one bin, both 21s go to B2 (6 values) and B3 becomes 22, 25, 25, 27 (4 values). Equal-width bins are uneven (5, 7, 4, 4) and sensitive to outliers; equal-frequency bins have equal counts.

### B3. Entropy-based discretization (best split point)

Labelled data: 10 N, 12 N, 15 N, 18 N, 22 Y, 25 Y, 28 N, 31 Y, 35 Y, 40 Y. There are 5 Y and 5 N, so Info(D) = $-0.5 \log_2 0.5 - 0.5 \log_2 0.5$ = **1.000**. For each candidate split s (midpoint between adjacent values): Info_s(D) = $(|D_1|/10)$ Entropy($D_1$) + $(|D_2|/10)$ Entropy($D_2$); Gain = Info(D) − Info_s(D).

| Split s | Left (≤ s) | Right (> s) | Entropy left / right | Info_s(D) | Gain | Rank |
|---|---|---|---|---|---|---|
| 11 | 1 N | 5 Y, 4 N (9) | 0 / 0.991 | 0.892 | 0.108 | 8–9 |
| 13.5 | 2 N | 5 Y, 3 N (8) | 0 / 0.954 | 0.764 | 0.236 | 5–6 |
| 16.5 | 3 N | 5 Y, 2 N (7) | 0 / 0.863 | 0.604 | 0.396 | 2–3 |
| **20** | 4 N | 5 Y, 1 N (6) | 0 / 0.650 | **0.390** | **0.610** | **1** |

| Split s | Left (≤ s) | Right (> s) | Entropy left / right | Info_s(D) | Gain | Rank |
|---|---|---|---|---|---|---|
| 23.5 | 4 N, 1 Y | 4 Y, 1 N (5) | 0.722 / 0.722 | 0.722 | 0.278 | 4 |
| 26.5 | 4 N, 2 Y | 3 Y, 1 N (4) | 0.918 / 0.811 | 0.876 | 0.125 | 7 |
| 29.5 | 5 N, 2 Y | 3 Y (3) | 0.863 / 0 | 0.604 | 0.396 | 2–3 |
| 33 | 5 N, 3 Y | 2 Y (2) | 0.954 / 0 | 0.764 | 0.236 | 5–6 |
| 37.5 | 5 N, 4 Y | 1 Y (1) | 0.991 / 0 | 0.892 | 0.108 | 8–9 |

Worked example for the best split s = 20: left = {10, 12, 15, 18} all N → entropy 0. Right = {22, 25, 28, 31, 35, 40} → 5 Y, 1 N: entropy = $-(5/6)\log_2(5/6) - (1/6)\log_2(1/6) = 0.650$. $\text{Info}_{20}(D) = (4/10)(0) + (6/10)(0.650) = 0.390$; Gain = 1 − 0.390 = **0.610**.

**Best split point = 20** (between 18 and 22), giving the intervals [10, 20) (pure N) and [20, 40] (mostly Y). The right interval could be split again (e.g. at 26.5/29.5) if the gain still exceeds the stopping criterion (MDL principle).

### B4. Comparison: unsupervised versus supervised discretization

| Aspect | Equal-width / equal-frequency (unsupervised) | Entropy-based (supervised) |
|---|---|---|
| Uses class labels? | No; only the value distribution. | Yes; chooses boundaries that maximise class purity (information gain). |
| Boundaries | Fixed by arithmetic (13.25, 23.5, … or counts), may cut through class-pure regions. | Placed where the class changes (20 here). |
| Quality on the labelled data | A 2-bin equal-width cut at 25 gives Info = 0.722 (each side is mixed). | The cut at 20 gives Info = 0.390 and a pure left interval. |
| Weaknesses | Equal-width is distorted by outliers; equal-frequency can split identical values. | Needs labels; may overfit; computationally heavier. |
| Best used for | Exploration, clustering, no labels available. | **Classification tasks.** |

**Conclusion:** for a classification task such as machine-failure detection, the supervised entropy-based discretization is more suitable, because it keeps the information about the class that the model must learn. Unsupervised binning is appropriate when no labels exist or for descriptive summaries.

# Q7 – Day-4 Design Challenge: Diagnose, Redesign and Validate (26 Marks)

**QUESTION**
A retail bank's loan-default pipeline: (1) missing income replaced by the global mean; (2) occupation (18% missing) dropped; (3) skewed income discretized into 5 equal-width intervals; (4) all numeric attributes min-max normalised on the complete dataset before the train/test split; (5) a random 10% sample used for training, with only 3% of loans being defaults. The model reported 97% accuracy but after deployment missed most defaulters.
A. Diagnose the defects (principle violated and how it misleads) – 2 marks. B. Define measurable design objectives and constraints – 2. C. Design an ordered revised pipeline, justify each step against an alternative and state whether it is computed on training data only – 4. D. Predict the effect on income dispersion, training class distribution and accuracy vs recall – 1. E. Implement original and redesigned pipelines on a public credit dataset and compare – 2. F. Compare predictions with results, defend the design and state one remaining risk – 2.

**ANSWER**

## A. Diagnose the pipeline (2 marks)

The real bottleneck is not computing power but information loss and an invalid evaluation: the pipeline throws away signal, leaks test information and measures the wrong metric. The 97% accuracy equals what a model that never predicts "default" would score (1 − 0.03), so it says nothing about the ability to find defaulters.

| # | Defect | Principle violated | How it misleads the model or evaluation |
|---|---|---|---|
| 1 | Global mean imputation of income (computed on all data) | Preserve the distribution; use robust, group-aware imputation; fit statistics on training data only | Income is right-skewed, so the mean is not typical. A spike of identical values at the mean shrinks variance (about 8% in the SD at 15% missing), weakens the income–default relationship, and hides whether income was missing. Using all rows also leaks test information. |
| 2 | Dropping occupation (18% missing) | Do not discard informative attributes because of missingness; handle missing values | Occupation is a likely predictor of repayment, so the model loses signal. Missingness may itself be informative ("unknown/unstable" work) and is lost. |
| 3 | 5 equal-width intervals for skewed income | Discretize according to the data distribution (equal-frequency or class-aware) | A few very high incomes stretch the width, so most customers fall into the first one or two bins and the other bins are almost empty. Risk differences among the bulk of customers disappear; extreme incomes dominate the bin edges. |
| 4 | Min-max normalisation on the complete dataset before the split | No data leakage: fit transformations on training data only and apply them to test data | The test set's minimum and maximum influence the training scaling, giving optimistic and unrealistic evaluation; in deployment, new data will not be known in advance, and out-of-range values break the [0, 1] assumption. Min-max is also very sensitive to outliers. |
| 5 | Random 10% training sample with 3% defaults | Sampling must preserve the rare class (stratified); do not shrink the minority class | Only about 3% of 10% of rows are defaulters, so the model sees very few (possibly none in a small sample). It learns to say "no default". Throwing away 90% of data also raises variance. |
| 6 | Evaluation by accuracy on 3% prevalence (random, unstratified test) | Choose metrics suited to the class distribution (recall, precision, PR-AUC, cost) | Accuracy of 97% is achieved by predicting "non-default" for everyone; this is the accuracy paradox. Failure to detect defaulters (recall near 0) stays invisible until deployment. |

## B. Design objectives and constraints (2 marks)

**Measurable objectives (evaluated on one untouched, stratified test set):**

- **O1 (primary):** raise recall on defaulters by at least **+0.25 absolute** over the original pipeline (original ≈ 0). The production target for the bank's full data is recall ≥ 0.60.

- **O2:** keep precision on flagged loans at least **2× the base rate** (≥ 6% when prevalence is 3%) so that the review workload stays manageable.
- **O3 (acceptable trade-off):** accept a fall in overall accuracy of up to **15 percentage points** (accuracy ≥ 0.80), because accuracy is not the business goal.
- **O4:** report PR-AUC, F1 and the spread over repeated splits (not a single lucky split).

**Constraints the redesigned pipeline must satisfy:**

- No leakage: every statistic (median, scaler, bin edges, selected features, class weights/resampling) is computed on training data only and then applied to the test and new data.
- Keep all information available: no dropping of attributes because of missingness alone; keep all defaulters in training.
- Reproducible and deployable: the same fitted preprocessing objects are applied to new loan applications.
- Interpretable enough for credit-policy and regulatory review (decision tree or logistic regression, not a black box).
- Computation within the bank's batch window; no manual steps; and fairness/privacy rules respected.

## C. Revised preprocessing strategy (4 marks)

| # | Step (technique class) | Technique chosen | Why chosen over an alternative | Fitted on training data only? |
|---|---|---|---|---|
| 0 | Split | Stratified 70/30 train/test split (and stratified k-fold CV inside training) **before any processing** | Random split may give very few defaults in the test set; stratification keeps 3% in both. Prevents leakage from the start. | Split itself; all later steps: **Yes** |
| 1 | Cleaning: missing income | Median imputation (preferably by occupation/loan-type group, not by default label) plus a **missing-indicator** column | Median is robust to skew, unlike the global mean; the indicator keeps the signal in missingness. Regression/kNN imputation is a good alternative but costlier and needs a missing-at-random assumption. | **Yes** (medians from training) |
| 2 | Cleaning: occupation (18% missing) | Keep the attribute; treat missing as its own category "Unknown" | Dropping loses information; mode imputation would hide the missingness; "Unknown" is simple and explainable. | Yes (category list) |
| 3 | Integration / redundancy check | Correlation (Pearson) for numeric and $\chi^2$ for nominal attributes; drop one of any pair with $\|r\| > 0.9$ or very strong association; check duplicates | Removes redundant attributes caused by merged sources without discarding information; PCA alternative reduces interpretability. | **Yes** |
| 4 | Transformation | Log transform of income, then z-score standardisation of numeric attributes; one-hot encoding for categories | Log removes right skew; z-score is less distorted by outliers than min-max and keeps outliers visible. Min-max alternative is outlier-sensitive. | **Yes** (mean/SD from training) |
| 5 | Discretization of income | **Equal-frequency (quantile) bins**, 5 bins on log-income (or entropy/MDLP bins if a tree/rule model is required) | Each bin holds about 20% of customers, so skew no longer empties bins; entropy-based bins are class-aware. Equal-width is rejected. | **Yes** (bin edges from training) |
| 6 | Reduction | Attribute subset selection (L1 regularisation or decision-tree induction inside cross-validation); keep all rows of the minority class | Keeps readable attributes; PCA rejected for interpretability. No blind 10% row sampling. | **Yes** |
| 7 | Class imbalance | **Cost-sensitive learning** (class weights); SMOTE or stratified under-sampling of non-defaults on training folds is the alternative | Uses all data without discarding information; synthetic samples can add noise in small data sets. Test data is never resampled. | **Yes** (training only) |

| # | Step (technique class) | Technique chosen | Why chosen over an alternative | Fitted on training data only? |
|---|---|---|---|---|
| 8 | Modelling and evaluation | Logistic regression / decision tree; tune decision threshold on validation folds; evaluate once on the untouched test set with recall, precision, F1, PR-AUC and confusion matrix | Accuracy is replaced by metrics that reflect the 3% prevalence and the cost of missed defaults. | Threshold tuned on training/validation only |

## D. Predictions made before implementation (1 mark)

| Quantity | Prediction | Reasoning |
|---|---|---|
| Dispersion of income | The original mean-filled income has a **smaller SD** than the true spread (about 8% less when 15% is missing) and a spike at the mean; the redesigned version keeps the observed spread, and after the log transform the skewness drops from strongly positive (> 1.5) to about 0. | Mean imputation adds zero-deviation points: variance falls by about (1 − m). Log compresses the long right tail. |
| Class distribution in the training data | Original: 10% sample, about 3% defaults but only a handful of cases (possibly 0–3). Redesigned: stratified, 3% defaults in all training data, but **effectively balanced 50/50 through class weights** (about 7× more training rows than the 10% sample). | Stratification preserves prevalence; weights make each default count about 32× a non-default. |
| Accuracy versus recall | **Accuracy will fall** (from about 96–97% to about 80–90%) while **recall on defaulters will rise** from near 0 to roughly 30–60%. Precision will stay low (below 15%). | The original model predicts almost only "no default"; the cost-sensitive model flags more loans, producing more false alarms and fewer misses. |

## E. Implementation and validation (2 marks)

**Dataset:** UCI Statlog **German Credit** (1,000 loans, 20 attributes; 700 good and 300 bad). Source: archive.ics.uci.edu/dataset/144 (CSV copy: github.com/jbrownlee/Datasets, german.csv). **Tool:** Python 3 with pandas and scikit-learn (script *q7_pipeline_comparison.py* submitted with this report). **Settings:**

- Credit *amount* (skewed, similar to income) is the income proxy; 15% of its values were made missing at random. Attribute *job* is the occupation proxy; 18% made missing.
- The default (bad) class was down-sampled to **3%** of rows (all 700 good loans + 22 defaults = 722 rows) to mimic the bank's data.
- Same stratified 70/30 test set is used for both pipelines. Because the test set has only about 7 defaults, results are averaged over **30 repeated random splits** (mean ± SD shown).
- **Original pipeline:** global-mean imputation, drop job, 5 equal-width bins, min-max on all data before the split, 10% random training sample, logistic regression (when the 10% sample contained no default, the model could only predict "no default").
- **Redesigned pipeline:** steps 0–8 above (median + indicator, job = Unknown, log + z-score, 5 equal-frequency bins, one-hot, logistic regression with class weights, C = 0.3), all fitted on the training part only.

| Metric (test set, mean ± SD over 30 splits) | Original pipeline | Redesigned pipeline | Objective met? |
|---|---|---|---|
| Accuracy | 0.961 ± 0.018 | 0.851 ± 0.034 | O3: yes (≥ 0.80) |

| Metric (test set, mean ± SD over 30 splits) | Original pipeline | Redesigned pipeline | Objective met? |
|---|---|---|---|
| Recall on defaulters | 0.019 ± 0.062 | 0.367 ± 0.179 | O1: yes (+0.35 ≥ +0.25) |
| Precision (defaulters) | 0.055 ± 0.202 | 0.089 ± 0.052 | O2: yes (≥ 0.06) |
| F1 (defaulters) | 0.022 ± 0.068 | 0.141 ± 0.075 | Improved |
| PR-AUC | 0.105 ± 0.080 | 0.130 ± 0.092 | Slight gain (within noise) |
| ROC-AUC | 0.604 ± 0.115 | 0.704 ± 0.113 | Improved |

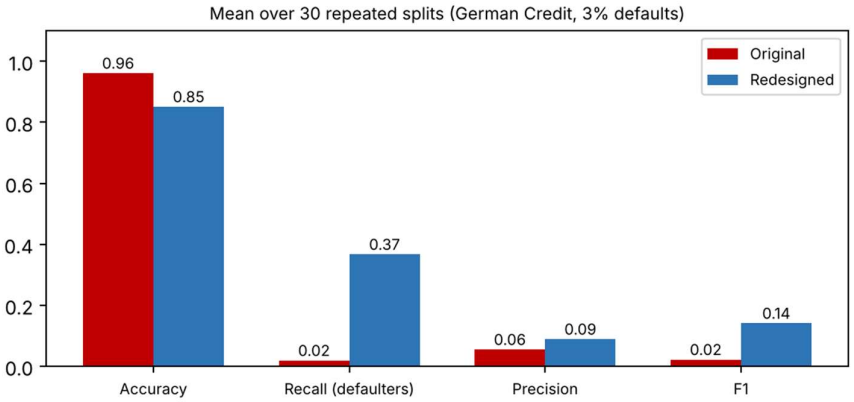



*Figure 3: Original versus redesigned pipeline (same test sets)*

**Dispersion check (averages over the 30 data sets):** SD of the observed credit amounts = 2,449; after global-mean imputation the SD = 2,252 (**−8.1%**), as predicted. Skewness of the amount falls from **1.96 to 0.05** after the log transform.

## F. Defence of the final design (2 marks)

**Predictions versus results.** All three predictions held: (1) income SD fell by about 8% under mean imputation (predicted about 8%) and skewness dropped to about 0; (2) the original 10% sample (about 72 rows) holds only about 2 defaults on average, so recall was about 0.02, while the redesigned model used all stratified training data with class weights; (3) accuracy fell from 0.961 to 0.851 and recall rose from 0.02 to 0.37 (in the predicted ranges), with precision remaining low (0.09).

**Defence.** The original pipeline's 96–97% accuracy was an artefact of the 3% prevalence. The redesign removes each defect: it avoids leakage (split first, fit on training only), retains information (median + indicator, "Unknown" occupation), respects the skewed distribution (log, equal-frequency bins), protects the minority class (stratification, class weights) and judges the model by recall, precision and PR-AUC. It meets objectives O1–O3 using a transparent logistic-regression model. Accuracy is deliberately sacrificed because missing a defaulter costs far more than reviewing an extra application.

**One remaining risk.** Missing income is unlikely to be missing at random in real life (applicants with low or irregular income may leave it blank), so median imputation plus an indicator may still bias the risk estimate; and in this experiment the test set contains only about 7 defaults, so the recall estimate is noisy (SD 0.18). The design therefore needs validation on the bank's full data and monitoring for data drift after deployment.

**Honest limitations.** The German Credit data has no real income or occupation fields and no missing values, so *amount* and *job* were used as proxies and the missingness and 3% prevalence were simulated. The gains in PR-AUC are within the spread between splits; the clear improvement is in recall.